

Curso Inicial Front-End

Clase 21:

Componentes y

Sintaxis JSX

Módulo 3: Desarrollo con React.js y Ecosistema Moderno



Presentación

El poder de **React** reside en su capacidad de crear piezas modulares. En esta clase profundizaremos en los **Componentes y Props**. Aprenderemos a pasar datos entre piezas de LEGO (Padre a Hijo) y dominaremos el renderizado dinámico de listas usando `.map()`, entendiendo por qué la reutilización de código es el secreto de las grandes aplicaciones.



Objetivos

- Dominar el paso de información mediante **Props** y la desestructuración profesional.
- Implementar el renderizado de listas dinámicas con `.map()` y el uso correcto de `key`.
- Comprender los beneficios de la arquitectura de componentes frente al **HTML** estático.



Bloques Temáticos

- Tema 1: Creación de componentes funcionales y pasaje de propiedades (Props).
 - ¿Qué es un Componente?
 - Renderizado Dinámico
 - Props: Argumentos para la Vista
 - El Superpoder: La Prop children
 - Renderizado de Listas: El Robot Repetidor
 - Ejercicio de Clase: Tu Primera Galería

En **React**, construimos aplicaciones como si armáramos un set de LEGO. No esculpimos una página web de un solo bloque gigante de mármol; ensamblamos pequeñas piezas independientes y reutilizables llamadas **Componentes**.

Y para que esas piezas hablen entre sí y se pasen información, utilizamos las **Props** (Propiedades).

 **Recurso Oficial:** [React Docs - Pasar Props a un Componente](#)

Tema 1: Creación de componentes funcionales y pasaje de propiedades (Props).

1. ¿Qué es un Componente?

En React, un **componente** es una pieza de código independiente y reutilizable que define una parte de tu interfaz (como un botón, una barra de navegación o un perfil de usuario). Imagina que el **HTML** tradicional es un **póster impreso**: una vez que sale de la imprenta, no puedes cambiar el precio de un producto sin imprimir un póster nuevo. **React** es una **pantalla digital interactiva**: está formada por bloques inteligentes que pueden cambiar su contenido en tiempo real.

| Beneficio: Reutilización vs Repetición

- **HTML Estático**: Si tienes 10 tarjetas de productos, tienes que escribir 10 veces el mismo código. Si quieres cambiar el color de todas, tienes que editar 10 lugares.
- **React**: Escribes el componente **Card** una sola vez y lo llamas 10 veces pasándole datos distintos. Si cambias el color en el componente, ¡los 10 cambian al instante!

Actualmente, existen dos formas de crearlos: los **funcionales** (el estándar moderno) y los de **clase** (el estilo antiguo). Aquí la diferencia:

Componente Funcional

Es simplemente una función de JavaScript que recibe datos (llamados **props**) y devuelve lo que debe verse en pantalla (el **JSX**).

- **¿Cómo se ve?** Es corto y directo.
- **¿Por qué es el estándar?** Gracias a los *Hooks* (Que los veremos en la próxima clase), ahora los componentes funcionales pueden manejar **memoria y lógica compleja** sin necesidad de una sintaxis complicada.

```
function Saludo(props) {  
  return <h1>Hola, {props.nombre}</h1>;  
}
```

Componente de Clase

Es una clase de **JavaScript** (basada en la **programación orientada a objetos**) que se extiende de `React.Component`.

- **¿Por qué es antiguo?** Requiere mucha más escritura (*boilerplate*), el uso de **this** (que suele confundir a los principiantes) y métodos de ciclo de vida que a veces resultan difíciles de gestionar.
- **¿Se sigue usando?** Sí, en proyectos antiguos (código heredado), pero ya no se recomienda para nuevos desarrollos.

```
class Saludo extends React.Component {  
  render() {
```

```
    return <h1>Hola, {this.props.nombre}</h1>;  
  }  
}
```

2. Renderizado Dinámico

En **React**, no usamos bucles `for` para mostrar listas. Usamos `.map()` porque devuelve un nuevo array de elementos visuales.

Truco Visual Importante: Cuando usamos `.map()` dentro de JSX, solemos usar **paréntesis** `()` en lugar de llaves `{}`. Esto se llama **Retorno Implícito**: le dice a **JS** "todo lo que hay aquí adentro es **HTML**", devuélvelo directamente".

```
{usuarios.map((user) => (  
  <div key={user.id}>  
    <h3>{user.nombre}</h3>  
  </div>  
))}
```

¿Por qué no usar `forEach`?

`forEach` solo recorre pero **no devuelve nada** (devuelve `undefined`). Como **React** necesita "ver" algo para dibujarlo en pantalla, `.map()` es la única opción válida porque genera la lista de componentes.

3. Props: Argumentos para la Vista

Si un componente es una función, las **Props** son los **Argumentos** que le pasamos a esa función. Imagina que hiciste el componente `TarjetaUsuario`. Si lo

usas 3 veces, no quieres que las 3 tarjetas digan "Perfil de Usuario" genérico. Quieres pasarle un nombre distinto a cada una.

¿Cómo se pasan las Props?

Desde el componente Padre (quien "invoca" al hijo), le pasas información como si fueran atributos **HTML** normales.

```
// El Padre (App.jsx)
// Llamamos al hijo 3 veces, pasándole una 'prop' distinta en cada
// invocación.
function App() {
  return (
    <>
      <TarjetaUsuario nombre="Lucas" edad={28} />
      <TarjetaUsuario nombre="Ana" edad={25} />
      <TarjetaUsuario nombre="Marta" edad={60} />
    </>
  );
}
```

¿Cómo las recibe el Hijo? (Destructuring)

El componente Hijo recibe un **Objeto** gigante que contiene todas esas propiedades. Para evitar código feo como `props.nombre`, aplicamos inmediatamente la técnica de "Destructuring" que aprendimos en **JS** avanzado (Regla 7).

```
// El Hijo (TarjetaUsuario.jsx)
// Extraemos 'nombre' y 'edad' de los paréntesis directamente.

function TarjetaUsuario({ nombre, edad }) {
  return (
    <div className="tarjeta">
      <h2>Perfil de: {nombre}</h2>
    </div>
  );
}
```

```
    <p>Tiene {edad} años.</p>
  </div>
);
}
```

🔴 **Checkpoint de Comprensión (La Ley de Hierro):** Las Props son de **Solo Lectura (Inmutables)** y el flujo de datos siempre viaja de Padre a Hijo (como una cascada). Si el componente Hijo intenta ser rebelde y ejecutar una línea como `nombre = "Juan Carlos"` para reescribir la Prop que su Padre le envió, ¿qué crees que hará **React**? (Pista: Piensa si el agua de una cascada puede fluir hacia arriba).

4. El Superpoder: La Prop `children`

A veces no quieres pasarle un simple texto a un componente. Imagina que quieres hacer un componente `CajaDecorada` y quieres meterle adentro cualquier cosa (una imagen, un texto, otro botón). **React** nos regala la prop mágica llamada `children`.

```
// El Hijo (Caja.jsx)

function CajaDecorada({ children }) {
  // Todo lo que el Padre inyecte dentro de la etiqueta Caja, caerá aquí.
  return <div className="borde-dorado-bonito">{children}</div>;
}

// El Padre (App.jsx)

function App() {
  return (
    <CajaDecorada>
      {/* Estos dos elementos son el "children" */}
      <h1>Título gigante</h1>
      <button>Botón interno</button>
    </CajaDecorada>
  );
}
```

```
    </CajaDecorada>  
  )  
}
```

5. Renderizado de Listas: El Robot Repetidor

Cuando pides datos a una Base de Datos (ej: una lista de 50 usuarios), no vas a escribir 50 componentes `<TarjetaUsuario />` a mano. Vas a usar tu viejo amigo: el método de array `.map()`.

⚠ IMPORTANTE

La Trampa del `.map()` (La prop `key`): Cuando haces un `.map()` para generar componentes repetidos, **React** te obligará a pasarle una prop especial llamada `key` con un ID único. **React** usa esta huella digital para saber exactamente qué componente borrar de la pantalla si un usuario de la lista es eliminado.

Ejemplo Visual:

```
const listaUsuarios = [  
  { id: 101, nombre: "Lucas" },  
  { id: 102, nombre: "Ana" }  
];  
  
function Lista() {  
  return (  
    <div>  
      {listaUsuarios.map((usuario) => (  
        // Observa el uso obligatorio de 'key'  
        <TarjetaUsuario key={usuario.id} nombre={usuario.nombre} />  
      ))}  
    </div>  
  );  
}
```

Documentación Oficial (Referencias)

- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **React:** <https://es.react.dev/>